

Session 4 — Study Guide: Redux Integration Patterns

This guide explains in depth what changed in B4F Hub in Session 4 and, more importantly, **why**. If you forgot most of the class, you should be able to read this alone and rebuild your understanding. The short version is in the Student Notes; this is the long version.

Before this session began, the Session 3 continuity homework (the recently-viewed migration to Redux) was reviewed and its official solution taught live, so everyone started today's new content with two working slices already in the store.

1. The correction this session opened with

Last session, in class, it was said that "Redux persists the state automatically, so we don't need to deal with localStorage." **That statement was wrong**, and it was corrected explicitly at the start of today, not softened or left alone.

The accurate model: `configureStore` builds a store that lives in a JavaScript variable, in memory, for as long as the page stays open. A full page reload runs the whole app from zero and rebuilds the store from `initialState` — every slice starts over. This is exactly the same as Context: neither ever persisted anything. If an app needs data to survive a reload, that is always a separate, deliberate decision — `localStorage`, `sessionStorage`, `IndexedDB`, a real backend, or (later, not built by hand) a library like Redux Persist. Later in this same session, after the teamwork, that decision is actually made for one piece of state — see section 8.

2. Where the session starts and ends

Start. Two Redux Toolkit slices: `savedOpportunities` (from Session 3) and `recentlyViewed` (from Session 3's homework, reviewed live today). `NotificationContext` is the only Context left. Three files still write `(state: RootState) => ...` by hand.

End. Every Redux consumer reads and writes through two small, typed hooks (`useAppDispatch` / `useAppSelector`). A new file, `store/selectors.ts`, holds selectors that genuinely need more than one slice. One small new UI feature — an "Activity" readout — shows a number sourced from both slices at once. Your team also added a derived selector and a control that resets both slices from one click. After the teamwork solution, `savedOpportunities.savedIds` — and only that piece of state — is wired to `localStorage`, so saved opportunities now survive a reload.

3. The question this session answers

Sessions 2 and 3 each introduced one new tool for sharing state (Context, then Redux Toolkit). This session introduces no new tool at all. Instead: **now that this app genuinely has more than one slice, how do components read them cleanly, and can one UI feature care about more than one slice at once?**

This is deliberately not "build a third slice." Repeating the create-a-slice pattern again would teach nothing new — you already did that, correctly, in the Session 3 homework. Today is about integration and judgment on the slices that already exist.

4. Typed hooks: what they actually buy

TS

```
// store/hooks.ts
import { useDispatch, useSelector } from "react-redux";
import type { TypedUseSelectorHook } from "react-redux";
import type { AppDispatch, RootState } from "./store";

export const useAppDispatch: () => AppDispatch = useDispatch;
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector;
```

At runtime, `useAppSelector(fn)` and `useSelector(fn)` are the identical function call. Nothing about how the app *behaves* changes. What changes is what TypeScript already knows: every `useAppSelector` call has `state` pre-typed as `RootState`, so a component never again writes `(state: RootState) => ...` by hand. Three files had that annotation before today; none do after.

`AppDispatch` was exported back in Session 3 and explicitly not used anywhere — "prepared for later use." Today is that later use. It is worth being honest that `useAppDispatch()` behaves identically to `useDispatch()` for every action in this app, because every action here is synchronous. `AppDispatch`'s real payoff arrives once this app ever needs a typed thunk — not yet, and not promised for a specific session.

5. Where does a cross-slice selector live?

Every selector written in Sessions 2 and 3 lived inside the one slice file it read. Session 4 has a selector that cannot honestly live in either slice file:

TS

```
// store/selectors.ts
export function selectTotalActivityCount(state: RootState) {
  return selectSavedCount(state) + selectRecentlyViewedCount(state);
}
```

This could be written inside `savedOpportunitiesSlice.ts`, reaching directly into `state.recentlyViewed.entries.length` — but that would make the saved-opportunities slice responsible for knowing the recently-viewed slice's internal shape, which is exactly the kind of coupling slices exist to avoid. The rule this session establishes: **a selector that needs only one slice's data lives in that slice's file; a selector that needs more than one slice's data lives in its own module, and composes each slice's own named selectors rather than reading their raw state.**

6. One UI feature, two slices, zero duplicated state

TSX

```
const totalActivity = useAppSelector(selectTotalActivityCount);  
// ...  
<p className="activity-summary">Activity: {totalActivity}</p>
```

From this component's point of view, `totalActivity` is just a number, read the same way `savedIds` is read one line above it. The component has no idea, and does not need to know, that the number underneath is really two different reducers' state added together. Nothing new was stored to make this work — `totalActivity` is entirely derived, recomputed from existing state every time it is read.

7. One handler, two slices, two dispatches

The teamwork's Reset activity control clears both slices from one click:

TSX

```
onClick={() => {  
  dispatch(clearAllSaved());  
  dispatch(clearRecentlyViewed());  
}}
```

`createSlice` scopes every reducer to its own slice's draft state — there is no mechanism for one action to be handled by two different slices' reducers. The component itself is the coordination point: it is free to dispatch as many actions as the situation genuinely needs, one per concern. This is not a workaround or a shortcut — it is the normal way independent slices are coordinated.

8. Persisting savedIds, manually, after the teamwork

Once the teamwork solution is demonstrated, this session closes the loop the opening correction opened: it adds real, working persistence — for exactly one piece of state.

The rule first, stated precisely: persist only `savedOpportunities.savedIds`. Not `recentlyViewed`, not `NotificationContext`, not `savedOnly`, not loading/error state, and not the whole store. That narrowness is itself the lesson — deciding what to persist is a product/state- ownership decision, made one piece of state at a time, never a blanket setting.

Restoring on startup. `localStorage` only stores strings, so a saved array has to be turned into one (`JSON.stringify`) and back (`JSON.parse`):

TS

```
// store/savedOpportunitiesSlice.ts
export const SAVED_OPPORTUNITY_IDS_KEY = "savedOpportunityIds";

function loadSavedIds(): number[] {
  const saved = localStorage.getItem(SAVED_OPPORTUNITY_IDS_KEY);

  if (!saved) {
    return [];
  }

  try {
    return JSON.parse(saved);
  } catch {
    return [];
  }
}

const initialState: SavedOpportunitiesState = {
  savedIds: loadSavedIds(),
};
```

No key yet, or a value that fails to parse, both fall back to `[]` — the same starting point as before this block existed. The `try / catch` exists because browser storage can outlive the code that wrote it (a stale format, manual tampering); it is one honest fallback, not schema validation.

Saving on every change. Reading state back is only half the loop — something has to write it, every time it changes:

TS

```
// store/store.ts
store.subscribe(() => {
  const state = store.getState();

  localStorage.setItem(
    SAVED_OPPORTUNITY_IDS_KEY,
    JSON.stringify(state.savedOpportunities.savedIds),
  );
});
```

`store.subscribe` runs *after* a state change already happened — it is a side effect, not a calculation, so it does not belong inside a reducer. Reducers only calculate the next state; a reducer that also wrote to `localStorage` would be doing two unrelated jobs at once.

The full loop, in both directions:

```

Redux state changes → store.subscribe() → JSON.stringify(...) → localStorage
localStorage → JSON.parse(...) → slice initialState → Redux store (on startup)

```

Rehydration is the name for the second line: rebuilding application state from previously persisted data when the app starts.

The precise distinction to hold onto: this is not "Redux persistence." Redux itself still does nothing differently — `configureStore` was not touched. What exists now is ordinary application code that happens to read from and write to `localStorage`. Say "we are persisting data used by Redux," never "Redux persists it."

A note on security and scope. `localStorage` is plain text, readable by any script on the same origin — it is browser-local persistence, not a secure database. This lesson stores a list of numeric ids; it never stores passwords, tokens, or anything else sensitive, and this session does not touch authentication at all.

Redux Persist, mentioned once and not used: a library that automates exactly this wiring — reading initial state from storage and writing it back on every change — for one or more slices, with more configuration than built here by hand. It is not installed or implemented this session; understanding the manual version first is what makes the automated version make sense later.

9. What genuinely did not change

`NotificationContext` is still Context, for the same reason it always has been: a transient UI queue read by one component has no need for action/reducer structure. `savedOnly` is still local `useState` in `OpportunitiesSection` — one panel's own UI state. No third slice was created anywhere this session or its homework. As of the end of this session, recently-viewed, notifications, `savedOnly`, and loading/error state all still reset on a reload — only `savedOpportunities.savedIds` was decided to persist, and that decision was made explicitly, not by default. This session's continuity homework later makes a second, equally explicit decision — persisting `recentlyViewed.entries` too, under its own separate key — as its CHALLENGE tier; notifications and other transient UI state never do, even after the homework.

10. What you should be able to explain now

- The corrected model: Redux (and Context) hold state in memory only; neither persists by default.
- What `useAppDispatch` / `useAppSelector` change, and what they explicitly do not change.
- How to decide whether a new selector belongs inside a slice file or in `store/selectors.ts`.
- Why `selectTotalActivityCount` calls `selectSavedCount` / `selectRecentlyViewedCount` instead of reading either slice's raw state directly.
- Why clearing two slices from one click is two `dispatch` calls, not a new combined reducer.
- Why `NotificationContext` is still a Context after two more Redux-focused sessions.
- The full save/restore loop for `savedIds`, in both directions, and why it lives in `store.subscribe` and a loader function, never inside a reducer.
- Why "we are persisting data used by Redux" is the correct phrase, and "Redux persists it" is not.
- Why only `savedIds` was persisted, and what would need to be true before persisting anything else.

11. Not yet

These exist and are **not** part of this session, and will not be needed until later in the course (most not until the Node/Express and NestJS blocks begin):

- `createAsyncThunk` and `extraReducers` — Redux with asynchronous work.
- RTK Query — data fetching built on Redux.
- `createSelector` (reselect) — memoized selectors.
- Custom middleware, listener middleware, normalized entities (`createEntityAdapter`).
- Redux Persist (the library) — this session built its manual equivalent by hand; the library that automates it is mentioned, never installed.
- `sessionStorage` and `IndexedDB` implementations — mentioned as alternatives, not built.

Glossary

Word	Meaning
Typed hook	A project-specific wrapper (<code>useAppDispatch</code> , <code>useAppSelector</code>) around a react-redux hook, pre-applied with this app's own types.
Cross-slice selector	A selector function that reads more than one slice of <code>RootState</code> .
Selector composition	A selector calling another named selector instead of duplicating its logic.
Derived selector	A selector computed entirely from other selectors' results, not from new state.
<code>RootState</code>	The full shape of the store — one key per registered slice.
<code>AppDispatch</code>	The type of this store's own <code>dispatch</code> function.
Rehydration	Rebuilding application state from previously persisted data when the app starts.
<code>store.subscribe</code>	Registers a callback that runs after Redux finishes processing each dispatch (not conditional on whether state actually changed) — a side effect, never a place to change state.
Persisting data used by Redux	The phrase this session uses instead of "Redux persistence," since Redux itself still persists nothing.

Review questions

1. What, exactly, changes at runtime when a component switches from `useSelector` to `useAppSelector` ? What does not change?
2. Why does `selectTotalActivityCount` live in `store/selectors.ts` instead of `savedOpportunitiesSlice.ts` ?
3. `selectHasAnyActivity` calls `selectTotalActivityCount` instead of repeating its arithmetic. What would go wrong, eventually, if it repeated the arithmetic instead?
4. The Reset activity button dispatches twice. Could a single new reducer do the same job in one dispatch? Why or why not?

5. Why was the opening correction necessary, and what is the accurate model it replaced an inaccurate one with?
6. Name one thing this session deliberately did not introduce, and the reason given for leaving it out.
7. Saved opportunities now survive a reload. Why is it still accurate to say "Redux does not persist anything by default"?
8. Why does the code that writes to `localStorage` live in `store.subscribe`, not inside the `toggleSaved` reducer?
9. Why does the loader function fall back to `[]` instead of letting a parse failure crash the app?